

Hello

[1, 1, -10, 3, -10, 3, -10, 1] -> 6
 ^ ^ ^

[1, 1, -10, -10, -10, -10, -10, -10]

Python

```
from collections import  
defaultdict
```

```
def longest_repeating_subarray_after_three_replacements(nums):  
    counts = defaultdict(int)  
    left = 0  
    max_freq = 0  
    best = 0  
    for right in range(len(nums)):  
        counts[nums[right]] += 1  
        max_freq = max(max_freq, counts[nums[right]])  
        while (right-left+1) - max_freq > 3:  
            counts[nums[left]] -= 1  
            left += 1  
        best = max(best, right-left+1)  
    return best  
  
nums = [1, 2, 2, 2, 3, 4, 5]
```

Use the sliding window method

Expand the window, then use two pointers, , add each number in a frequency map, id also track the most frequent number, find the max frequency of each number., and then check how many replacements are needed, if the window size is like 5 and the most common number is 3 times, replacements needed would be 5-3=2, so we can basically change the other 2 numbers to match, so that would be a valid window , we gotta shrink if it's needed, if replacements needed until window valid again, keep the best answer for every window we update the maxc length that we saw so far,,

```
left= 0  
max_freq=2  
best=2
```

```
right=3  
counts={}
```

bojia@google.com